

Centralized Load, Skewed Results: The Need for Distributed Benchmarking in Modern Databases

Gunnar Antoni Solli Olsen

*Universitetet i Tromsø
Tromsø, Norway
guols6589@uit.no*

Jens Christian Valen Leynse

*Universitetet i Tromsø
Tromsø, Norway
jeley4465@uit.no*

Abstract

This study was originally intended to be a comparative analysis of the scalability and performance characteristics of YugabyteDB, a distributed database system that supports both distributed SQL (YSQL) and the Cassandra Query Language (YCQL), comparing it to Apache Cassandra, a widely-used NoSQL database known for its high availability and masterless architecture. Using the Yahoo! Cloud Serving Benchmark (YCSB), we evaluate both databases under a range of cluster sizes (1, 2, 4, 8, and 16 nodes) and workloads that reflect common read-write access patterns. In the comparison, the study reflects on the need for a distributed database benchmark for testing scalable distributed database systems.

I. INTRODUCTION

Modern applications increasingly demand distributed databases that combine horizontal scalability with strong consistency guarantees. While NoSQL systems like Apache Cassandra achieve scale through eventual consistency, newer distributed SQL databases like YugabyteDB aim to provide transactional guarantees without sacrificing performance.

Using the Yahoo! Cloud Serving Benchmark (YCSB), this study compares the architectural approaches of the two distributed database systems. We assess how these systems handle increasing cluster sizes under standardized read-write workloads, with particular attention to the benchmarking methodology's suitability for distributed environments.

II. BACKGROUND

A. Cassandra

Apache Cassandra is an open-source distributed NoSQL database that combines Amazon Dynamo's replication techniques with Google Bigtable's data and storage engine model. Designed initially at Facebook to address scalability limitations of relational systems, Cassandra implements a partitioned wide-column store with eventual consistency, optimized for global availability, and low-latency operations. Its architecture avoids relational

features like distributed joins and multi-partition transactions in favor of predictable performance at scale [3].

The database organizes data through Cassandra Query Language (CQL), using keyspaces (replication units), tables (partitioned column families), and rows (partition-key identified data). Key features include flexible schema, storage-attached indexing, and single-partition lightweight transactions, while deliberately excluding distributed joins and cross-partition operations. Operational tools like nodetool and atomic snapshots support cluster management, with configuration managed via `cassandra.yaml` and runtime adjustments [3].

B. Yugabyte

YugabyteDB is a high performance distributed SQL database designed to support large-scale, globally distributed applications. The architecture of YugabyteDB integrates a high-performance document storage layer, distributed consensus-based replication at the shard level, and support for multi-shard ACID transactions. This enables YugabyteDB to efficiently handle both workloads with low query latency and high-throughput OLTP use cases. YugabyteDB includes data replication via raft to ensure fault tolerance. As a cloud native solution, it offers flexible deployment options, including public/private clouds and Kubernetes-managed environments.

YugabyteDB uniquely reuses PostgreSQL's native query layer, which allows it to inherit the advanced RDBMS capabilities included in PostgreSQL. This architectural design allows it to utilize PostgreSQL's features like stored procedures, triggers, and user-defined types, while layering it atop YugabyteDB's own distributed DocDB storage layer. This approach also ensures that it inherits PostgreSQL's query optimization and parsing logic. [4]

YugabyteDB is ideal for fast-growing cloud-native applications, as it ensures business critical data remains highly available, consistent, and resilient even in the face of node failures. It provides strong guarantees against data loss while maintaining low-latency access. [2] YugabyteDB uses two querying languages that are compatible with Cassandra's CQL, and PostgreSQL's SQL, where existing queries are fully compatible with YCQL and YSQL [2] in order to allow easier migration from current systems using PostgreSQL or Cassandra.

C. Consistency Trade-offs

Hybrid architectures delegate consistency handling to the application layer, whereas YugabyteDB internalizes these decisions. Table I summarizes key differences:

TABLE I: CONSISTENCY MODELS ACROSS DATABASES

Model	PostgreSQL	Cassandra	YugabyteDB
Strong	✓	*	✓
Bounded Staleness	×	×	✓
Eventual	×	✓	×

*Via QUORUM at performance cost

III. EXPERIMENTAL SETUP

A. Hardware

The experiments are done using 16 Microsoft Azure D4s_v3 virtual machines, each instance has access to 4 virtual CPUs with 8 gb ram. Every virtual machine has access to 128 gb of ssd storage. All networking is done through an Azure virtual network (subnet), allowing the nodes to communicate within a local network.

B. Benchmarking tool

Yahoo! Cloud Serving Benchmark is a tool made to benchmark distributed database systems such as Apache cassandra, Amazon DynamoDB, and MongoDB. The tool includes multiple types of benchmarks designed to test different aspects of the distributed database system, while collecting metrics such as throughput and latency. In our experiment YCSB was run in a virtual machine using the same hardware as the database nodes, using 64 threads to emulate multiple users interacting with the system simultaneously. Each test was configured with a record count of 1,000,000 and an operation count of 1,000,000, ensuring consistency across all runs. The benchmark uses CQL, the Cassandra Query Language, which is specific to Cassandra-based systems.

C. Replication factor

The replication factor was adjusted to match the number of nodes for configurations with 1 and 2 nodes, ensuring that all data was replicated across all available nodes. For configurations with 4, 8, 12, and 16 nodes, the replication factor remained at the default value of 3, which is commonly used in distributed database setups to balance data redundancy and performance.

IV. RESULT

For both YugabyteDB and Cassandra, throughput increases as the number of nodes increases, but the rate of improvement decreases as more nodes are added. Cas-

sandra achieves higher throughput at lower node counts, particularly at 1, 2, and 4 nodes, but its throughput fluctuates slightly across runs. After 8 nodes, both YugabyteDB and Cassandra experience a drop in throughput, suggesting potential bottlenecks or resource saturation.

TABLE II: YUGABYTEDB AVERAGE THROUGHPUT AND STANDARD DEVIATION ACROSS NODES (64 THREADS)

Nodes	Average Throughput	Standard Deviation
1	11928.80	395.38
2	12389.53	185.72
4	19836.24	456.80
8	20261.71	534.35
12	16477.78	3693.71
16	17341.59	5362.73

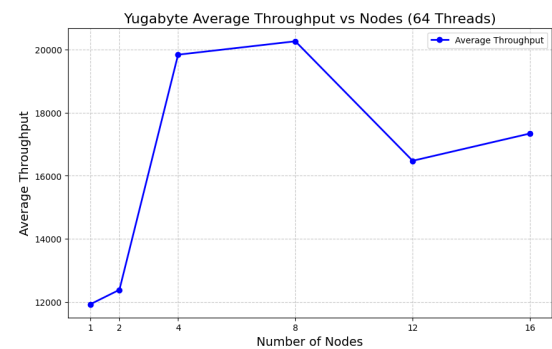


FIGURE I: YUGABYTEDB AVERAGE THROUGHPUT VS NODES.

TABLE III: CASSANDRA AVERAGE THROUGHPUT AND STANDARD DEVIATION ACROSS NODES (64 THREADS)

Nodes	Average Throughput	Standard Deviation
1	17683.50	1122.37
2	16474.38	761.74
4	26733.28	801.78
8	25711.66	540.60
12	25591.75	3312.82
16	24572.85	3724.81

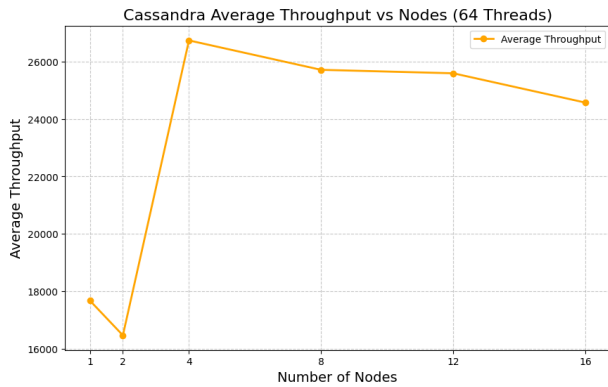


FIGURE II: CASSANDRA AVERAGE THROUGHPUT VS NODES.

As the number of nodes increases, the standard deviation of throughput values grows significantly for both YugabyteDB and Cassandra, showing more variability in performance at higher node counts. For YugabyteDB, the standard deviation remains relatively low and stable at smaller node counts, such as 1, 2, and 4 nodes. However, at 12 and 16 nodes, the standard deviation increases noticeably, suggesting that throughput becomes less consistent as the cluster grows.

Cassandra shows a similar trend, with standard deviation values staying relatively low at smaller node counts. As the cluster scales to higher node counts, the standard deviation increases sharply, indicating more variability in throughput.

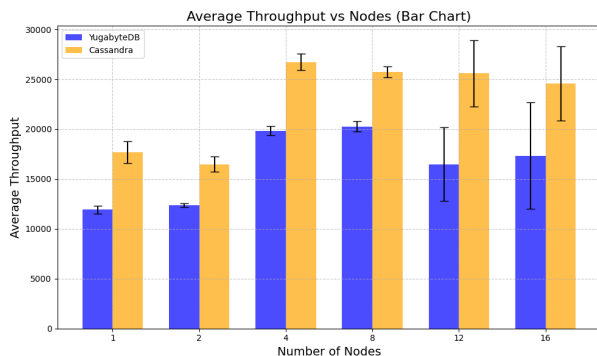


FIGURE III: DB AVERAGE AND DEVIATION

V. DISCUSSION

The databases show similar patterns when measuring throughput scalability, but YugabyteDB consistently has 21% to 35% lower throughput than Cassandra. For real world systems this would have an impact on the amount of nodes used in the system, as you would need more nodes in the database cluster when running YugabyteDB. In the experiments both database systems have the same replication factor of 3, the consistency level used for the experiments was 'QUORUM' for Cassandra, and the default consistency level of 'ONE' for YugabyteDB. This should

have given YugabyteDB an advantage for the tests with higher replication, but this advantage is not visible in the experiments.

The hardware used for the YCSB virtual machine is a possible bottleneck, there is a clear point on both figure I and figure 2II beyond 27000 ops/s where neither database shows any increased throughput, this is most likely due to the benchmarking node reaching its hardware limits before the virtual machines running each database node reach theirs. The ratio between hardware included in the distributed databases grows from 1:1 to 1:16. While YCSB was made to benchmark databases, as provided it becomes limited compared to horizontally scalable databases. This finding does not match the experiment conducted in the YCSB paper: "We observed in our tests that the client machine was not a bottleneck; in particular, the CPU was almost idle as most time was spent waiting for the database system to respond." [1]. Their experiment was conducted with hardware that had 8 cores per database node, using another 8 core system, in a non-cloud environment, with a far larger amount of threads used when running the benchmark (500 vs. our 64 threads). Since we never checked the utilization of our benchmarking node with the assumption that adding threads would lead to simultaneous nodes competing for CPU cycles, the benchmarking node probably could have used a higher number of threads without running into the hardware limits. As can be seen in figure III the throughput variability increases significantly for 12 and 16 nodes, this could have multiple causes, finding the true hardware performance ceiling, where threads become limited by CPU thread contention could be one cause, another possible bottleneck is the Azure virtual network that was shared by the database cluster and benchmarking node.

VI. CONCLUSION

Modern distributed databases are made to handle extreme amounts of traffic, this scale must be considered when benchmarking them. Both Cassandra and YugabyteDB outscale YCSB when the database node counts surpass a 4 to 1 ratio of database nodes compared to the node running the benchmark. In hindsight, this is an obvious bottleneck, but it provides a useful ratio that should be considered in future benchmarking experiments. Despite YCSB being designed for benchmarking distributed databases, the centralized nature of this benchmark makes it inherently limited for testing modern distributed databases such as Cassandra and YugabyteDB. We recommend that future database benchmarking tools should include functionality for distributing the benchmarking workload across multiple machines, before aggregating the result, this would simplify the process of benchmarking scalable databases, and would enable stress testing of elastic scalability without necessitating custom scripting to run multiple instances of the benchmarking program.

While both systems show similar scaling trends, their internal architectures diverge significantly. Cassandra favors a masterless, eventually consistent model optimized for high write throughput, while YugabyteDB offers distributed ACID compliance and a PostgreSQL-compatible SQL layer, trading throughput for stronger consistency and relational capabilities. Overall, given equal hardware, Cassandra is faster in every case, even while using a more strict consistency level.

CONFLICT OF INTEREST

The authors declare no conflict of interest.

REFERENCES

- [1] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, R. Sears. "Benchmarking cloud serving systems with YCSB". ACM.org <https://doi.org/10.1145/1807128.1807152>
- [2] YugabyteDB Documentation. *YugabyteDB Official Documentation*. Yugabyte, Inc., 2023. [Online]. Available: <https://docs.yugabyte.com/> [Accessed: 2025/04/01].
- [3] Apache Software Foundation, "Cassandra Architecture Overview," 2023. [Online]. Available: <https://cassandra.apache.org/doc/5.0/cassandra/architecture/overview.html>. [Accessed: 2025/04/01].
- [4] Yugabyte, Inc., "Why We Built YugabyteDB by Reusing the PostgreSQL Query Layer," *YugabyteDB Blog*, 2023. [Online]. Available: <https://www.yugabyte.com/blog/why-we-built-yugabytedb-by-reusing-the-postgresql-query-layer/>. [Accessed: 2025/04/23].